

Notacja asymptotyczna. Algorytmy dziel i zwyciężaj – Wykład

4 marca 2026

1 Notacja asymptotyczna

$f, g: \mathbb{N} \rightarrow \mathbb{R}_+$ (rosnąca).

- $g(n) = O(f(n))$ jeśli $\exists c > 0 \exists n_0 \forall n \geq n_0 \quad g(n) \leq c \cdot f(n)$
- $g(n) = \Omega(f(n))$ jeśli $\exists c > 0 \exists n_0 \forall n \geq n_0 \quad c \cdot f(n) \leq g(n)$
- $g(n) = \Theta(f(n))$ jeśli $\exists c_1, c_2 > 0 \exists n_0 \forall n \geq n_0 \quad c_1 f(n) \leq g(n) \leq c_2 f(n)$

Przykłady:

$$\begin{array}{ll} n^a = O(n^b), & 0 < a \leq b \\ \log n = O(n^\varepsilon), & \varepsilon > 0 \\ a^n = O(b^n), & 1 < a \leq b \\ n^a = O(b^n), & a > 0, b > 1 \\ \log_a n = O(\log_b n), & a, b > 1 \end{array}$$

Przykład: $T(n) = 5n^3 + 2n^3 + 100n \cdot \log n \approx 5n^3 = \Theta(n^3)$.

Sortowanie = $O(n \log n)$ – merge sort.

2 Procedura rekurencyjna

Przykład 1 (Silnia). $n!$:

$$\begin{cases} 0! = 1 \\ n! = n \cdot (n-1)! \quad \text{dla } n \geq 1 \end{cases}$$

$n! = n \cdot (n-1) \cdot \dots \cdot 1$.

fact(n):

1. **if** $n = 0$ **return** 1
2. **return** $n \cdot \text{fact}(n-1)$

Przykład 2 (Ciąg Fibonacciego). F_n : $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ dla $n \geq 2$.

1. **if** $n = 0$ **return** 0; **if** $n = 1$ **return** 1
2. **return** $\text{Fib}(n-1) + \text{Fib}(n-2)$

Wywołanie rekurencyjne dla tego samego problemu – nie dla podobnego, a tego samego – często dla mniejszych problemów.

3 Algorytmy dziel i zwyciężaj

Divide & conquer.

Algorytm typu dziel i zwyciężaj składa się z 3 części:

1. **Dziel** – problem jest dzielony na mniejsze podproblemy.
2. **Zwyciężaj** – rekurencyjnie rozwiązujemy te podproblemy.
3. **Połącz** – budujemy rozwiązanie całego problemu z mniejszych instancji.

Zazwyczaj zakładamy, że:

1. W fazie dziel problem jest dzielony na conajmniej dwa podproblemy.
2. Zakładamy, że te podproblemy są rozłączne (nie tak jak w rekurencji wyżej).

3.1 Mnożenie liczb n -cyfrowych

$x, y \in \mathbb{N}$ – n -cyfrowe.

Aby policzyć $x \cdot y$ musimy wykonać $O(n^2)$ mnożeń cyfr.

→ Znajdźmy lepsze rozwiązanie.

Dzielimy x i y na “dwie połowy”:

$$x = x_1x_2 \dots x_n = \underbrace{x_1x_2 \dots x_{n/2}}_{x_L} \underbrace{x_{n/2+1} \dots x_n}_{x_R} = 10^{n/2}x_L + x_R$$

$$y = 10^{n/2}y_L + y_R$$

$$x \cdot y = (10^{n/2}x_L + x_R)(10^{n/2}y_L + y_R) = 10^n x_L y_L + 10^{n/2}(x_L y_R + x_R y_L) + x_R y_R$$

$T(n)$ – złożoność czasowa tego algorytmu.

Podział x i y zajął nam $O(n)$ czasu. Dodawanie n -cyfrowych liczb $O(n)$. Mnożenie przez 10^n jest równoważne dopisaniu n zer, więc też zajmuje $O(n)$.

$$T(n) = 4 \cdot T\left(\frac{n}{2}\right) + O(n)$$

→ Można lepiej to zrobić:

$$x_L y_R + x_R y_L = (x_L - x_R) \cdot (y_R - y_L) + x_L y_L + x_R y_R$$

W tym rozwiązaniu zamiast 4 musimy policzyć tylko 3 iloczyny: $x_L y_L$, $x_R y_R$, $(x_L - x_R)(y_L - y_R)$.

Zatem

$$T(n) = 3 \cdot T\left(\frac{n}{2}\right) + O(n)$$

4 Uniwersalne twierdzenie o rekurencji (Master Theorem)

Twierdzenie 1 (Uniwersalne twierdzenie o rekurencji). Niech $a \geq 1$, $b > 1$ będą stałymi, $f(n)$ funkcją i

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

(gdzie $\frac{n}{b}$ znamy $\lfloor \frac{n}{b} \rfloor$ lub $\lceil \frac{n}{b} \rceil$) – ten ciąg podzieliśmy na a problemów $\frac{n}{b}$.

Wtedy

$$T(n) = \begin{cases} O(n^{\log_b a}) & \text{jeśli } f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0 \\ \Theta(n^{\log_b a} \cdot \log n) & \text{jeśli } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{jeśli } f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0, a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n), c < 1 \text{ dla dużych } n \end{cases}$$

Zanim udowodnimy to, udowodnimy lemat.

Lemat 1. Niech $a \geq 1$, $b > 1$ będą stałymi i niech $f(n)$ będzie nieujemną funkcją dla potęgi b .
Jeśli

$$T(n) = \begin{cases} \Theta(1) & n = 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & \text{dla } n = b^i, i \geq 1 \end{cases}$$

to

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Dowód.

$$\begin{aligned} T(n) &= f(n) + a \cdot T\left(\frac{n}{b}\right) = f(n) + a\left(f\left(\frac{n}{b}\right) + a \cdot T\left(\frac{n}{b^2}\right)\right) = \dots \\ &= f(n) + a f\left(\frac{n}{b}\right) + a^2 \cdot T\left(\frac{n}{b^2}\right) = \dots \\ &= f(n) + a f\left(\frac{n}{b}\right) + a^2 f\left(\frac{n}{b^2}\right) + a^3 \cdot T\left(\frac{n}{b^3}\right) = \dots \\ &= f(n) + a f\left(\frac{n}{b}\right) + \dots + a^{\log_b n - 1} f\left(\frac{n}{b^{\log_b n - 1}}\right) + a^{\log_b n} \cdot T(1) \end{aligned}$$

Ponieważ $a^{\log_b n} = b^{\log_b n \cdot \log_b a} = b^{\log_b n \cdot \log_n a \cdot \log_b n / \log_b n} = \dots = n^{\log_b a}$, stąd:

$$a^{\log_b n} \cdot T(1) = n^{\log_b a} \cdot T(1) = \Theta\left(n^{\log_b a}\right)$$

więc

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

□

4.1 Dowód twierdzenia (dla n będącego potęgą b)

Reszta też prawdziwa, ale dowód trudniejszy.

Rozważamy 3 przypadki:

Przypadek 1. $f(n) = O(n^{\log_b a - \varepsilon})$.

Wtedy $f\left(\frac{n}{b^i}\right) = O\left(\left(\frac{n}{b^i}\right)^{\log_b a - \varepsilon}\right)$.

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) &= \sum_{j=0}^{\log_b n - 1} a^j O\left(\left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) = O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) \\ &= n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{b^{j(\log_b a - \varepsilon)}} = n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{b^{j\varepsilon}}{b^{j \log_b a}} \cdot a^j \\ &= n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{a \cdot b^\varepsilon}{b^{\log_b a}}\right)^j = n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j \end{aligned}$$

(skończony szereg geometryczny)

$$= n^{\log_b a - \varepsilon} \cdot \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1} = n^{\log_b a - \varepsilon} \cdot \frac{n^\varepsilon - 1}{b^\varepsilon - 1} = n^{\log_b a - \varepsilon} \cdot O(n^\varepsilon) = O\left(n^{\log_b a}\right)$$

Korzystając z lematu:

$$T(n) = \sum a^j f\left(\frac{n}{b^j}\right) + \Theta\left(n^{\log_b a}\right) = O\left(n^{\log_b a}\right) + \Theta\left(n^{\log_b a}\right) = O\left(n^{\log_b a}\right)$$

Przypadek 2. $f(n) = \Theta(n^{\log_b a})$.

Wtedy $f(\frac{n}{b^i}) = \Theta((\frac{n}{b^i})^{\log_b a})$.

$$\sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j}) = \sum_{j=0}^{\log_b n - 1} a^j \Theta((\frac{n}{b^j})^{\log_b a}) \stackrel{\text{podobne obliczenia}}{=} \Theta(n^{\log_b n \log_b a})$$

Korzystamy z lematu:

$$T(n) = \sum a^j f(\frac{n}{b^j}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a \cdot \log_b n}) + \Theta(n^{\log_b a}) = \Theta(n^{\log_b a \log_b n})$$

Przypadek 3. $f(n) = \Omega(n^{\log_b a + \varepsilon})$ i $a \cdot f(\frac{n}{b}) \leq c \cdot f(n)$ dla $c < 1$ i dużych n .

$f(\frac{n}{b}) \leq \frac{c}{a} f(n)$, więc $f(\frac{n}{b^i}) \leq (\frac{c}{a})^i \cdot f(n) = \frac{c^i}{a^i} f(n)$.

$$\sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j}) \leq \sum_{j=0}^{\log_b n - 1} a^j \cdot \frac{c^j}{a^j} f(n) + O(1) = f(n) \cdot \sum c^j + O(1) \leq f(n) \cdot \sum_{j=0}^{\infty} c^j + O(1)$$

(nieskończony ciąg geometryczny, $c < 1$)

$$= f(n) \cdot \frac{1}{1-c} + O(1) = O(f(n))$$

Korzystamy z lematu:

$$\sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j}) = a^0 f(\frac{n}{b^0}) + \sum_{j=1}^{\log_b n - 1} a^j f(\frac{n}{b^j}) = f(n) + (\geq 0) = \Omega(f(n))$$

$$T(n) = \sum a^j f(\frac{n}{b^j}) + O(n^{\log_b a}) = O(f(n)) + \Theta(n^{\log_b a}) = \Theta(f(n))$$

Wniosek 1. Niech $a \geq 1$, $b > 1$ będą stałymi, $a, f(n)$ funkcją taką, że $f(n) = \Theta(n^k)$.

Niech $T(n) = a \cdot T(\frac{n}{b}) + f(n)$. Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{jeśli } a > b^k \iff \log_b a > k \\ \Theta(n^{\log_b a \log_b n}) & \text{jeśli } a = b^k \iff \log_b a = k \\ \Theta(n^k) & \text{jeśli } a < b^k \iff \log_b a < k \end{cases}$$

4.2 Powrót do mnożenia liczb n -cyfrowych

Drugi algorytm:

$$T(n) = 4 \cdot T(\frac{n}{2}) + O(n), \quad a = 4, b = 2, k = 1, \quad 4 > 2^1 \implies T(n) = \Theta(n^{\log_2 4}) = n^2$$

Trzeci algorytm:

$$T(n) = 3 \cdot T(\frac{n}{2}) + O(n), \quad a = 3, b = 2, k = 1, \quad 3 > 2^1 \implies T(n) = \Theta(n^{\log_2 3}) = n^{1,59}$$